



TRABAJO PRÁCTICO **INTEGRADOR**

**Estructuras de Datos, Contrato Equals/HashCode y
Complejidad Algorítmica**

Integrantes:

- Lacia Yanela
- Llerena Romina
- Soloa María
- Urbani Constanza
- Villarroel Lautaro

Comisión 3K10



Consigna 1: Diagnóstico de Bug Silencioso y Análisis de Memoria

Un equipo de desarrollo junior implementó el siguiente sistema para gestionar el acceso de empleados a un edificio mediante tarjetas de identificación mutable. El código no lanza excepciones en tiempo de ejecución, pero presenta comportamientos anómalos en producción.

Responder y Justificar:

1. Trazo del Estado Interno: Explique paso a paso qué ocurre dentro de la estructura HashSet:

- Al ejecutar `tarjetasValidas.add(t1)`: El HashSet toma los datos de la tarjeta `t1` (código "ACC-101" y nivel 2), ejecuta la función `hashCode()` para calcular en qué bucket le corresponde ubicarse, y guarda la referencia del objeto en ese cajón de la memoria.
- Al ejecutar `t1.setNivelAcceso(5)`: Como la tarjeta es un objeto mutable, se modifica su atributo cambiando el nivel a 5. Esto provoca que la fórmula del `hashCode()` interno de la tarjeta arroje un número completamente distinto, pero la tarjeta sigue físicamente estancada en el casillero viejo donde se guardó al principio, porque la estructura no se entera automáticamente de que cambió.
- Al ejecutar `tarjetasValidas.contains(t1)`: El sistema vuelve a calcular el `hashCode()` de la tarjeta, pero ahora con su nuevo estado (nivel 5). Con ese nuevo hash, Java va a buscar al cajón correspondiente a ese nuevo valor, y como allí no hay nada (la tarjeta sigue en el casillero anterior), el método devuelve `false`, aunque la tarjeta siga adentro del contenedor.

2. Explicación del Cajón (Bucket): ¿En qué número/ubicación de cajón quedó guardado el objeto y en cuál lo busca Java al hacer `contains()`?

El objeto quedó guardado físicamente en el cajón original calculado con el nivel 2 (cuando se hizo el `add`).

Sin embargo, al invocar el `contains()`, Java lo busca en el nuevo cajón calculado con el nivel actualizado (5).

3. Fuga de Memoria (Memory Leak): ¿Por qué este patrón genera una fuga de memoria si se ejecuta repetidamente en un servidor que opera 24/7?

Este patrón genera una fuga de memoria porque la tarjeta modificada queda "atrapada" dentro de la estructura del HashSet. Aunque el sistema devuelva `false` al buscarla y sea imposible acceder a ella de forma lógica, la estructura de datos sigue manteniendo una referencia activa a ese objeto en la memoria RAM.

Como sigue referenciado, el Recolector de Basura (*Garbage Collector*) no puede liberarlo. Si este proceso se repite miles de veces en un servidor que opera 24/7, la memoria se irá saturando progresivamente de objetos "fantasmas" inalcanzables hasta provocar un fallo general por falta de memoria (`OutOfMemoryError`).



4. Refactorización Obligatoria: Aplique dos soluciones arquitectónicas distintas para corregir el problema (una basada en inmutabilidad y otra ajustando el criterio de la entidad).

- Solución A (Basada en Inmutabilidad): Se elimina el método `setNivelAcceso` y se definen los atributos como final para que la clase sea inmutable. Si en el sistema se requiere cambiar el nivel de acceso de una credencial, no se modifica la tarjeta existente; en su lugar, se remueve la tarjeta vieja del Set y se crea e inserta una nueva instancia con el nivel actualizado.
- Solución B (Ajustando el criterio de la entidad / Separando identidad de estado): Se permite que la clase siga siendo mutable (conservando el setter), pero se excluyen los campos mutables (como `nivelAcceso`) del cálculo de `hashCode()` y `equals()`. De este modo, la identidad del objeto y su ubicación en los cajones del HashSet pasan a depender exclusivamente de un atributo fijo e inalterable (como el código), permitiendo que el nivel de acceso varíe libremente sin romper la estructura de hash.

Consigna 2: Selección y Diseño de Arquitectura de Colecciones

Requerimiento	Interfaz Seleccionada	Implementación Concreta	Complejidad de Búsqueda	Complejidad de Inserción	Justificación Técnica de la Elección
1. Auditoría	List	ArrayList	$O(n)$	$O(1)$	ArrayList permite inserciones eficientes al final en $O(1)$ y mantiene el orden de inserción estricto. Ofrece excelente rendimiento en recorridos secuenciales por localidad de memoria en caché.
2. Ofertas	NavigableSet	TreeSet	$O(\log n)$	$O(\log n)$	TreeSet mantiene los elementos ordenados automáticamente mediante un árbol. Permite consultar el producto más barato (<code>first()</code>) y el más caro (<code>last()</code>) en tiempo $O(1)$ u $O(\log n)$.



Requerimiento	Interfaz Seleccionada	Implementación Concreta	Complejidad de Búsqueda	Complejidad de Inserción	Justificación Técnica de la Elección
3. Sesiones	Map	HashMap	O(1)	O(1)	HashMap ofrece acceso directo instantáneo a través de claves únicas (Token) con complejidad O(1) constante promedio, ideal para buscar entre 500,000 usuarios concurrentes.

Consigna 3: Optimización Algorítmica ($O(n^2)$ -> $O(n)$)

Responder y Justificar:

1. Refactorización a $O(n)$: Escriba una versión optimizada del método utilizando la colección adecuada de java.util (HashMap, LinkedHashMap, o HashSet) que resuelva el problema realizando como máximo dos recorridos lineales ($O(n)$).

```
public static Character primerCaracterNoRepetido(String texto) {
```

```
    Map<Character, Integer> frecuencias = new HashMap<>();
```

```
    // Primer recorrido: contar frecuencias
```

```
    for (int i = 0; i < texto.length(); i++) {  
        char c = texto.charAt(i);  
        frecuencias.put(c, frecuencias.getOrDefault(c, 0) + 1);  
    }
```

```
    // Segundo recorrido: encontrar el primero con frecuencia 1
```

```
    for (int i = 0; i < texto.length(); i++) {  
        char c = texto.charAt(i);  
  
        if (frecuencias.get(c) == 1) {  
            return c;  
        }  
    }
```



```
return null;  
}
```

La primera pasada cuenta las apariciones utilizando un **HashMap<Character,Integer>**. La segunda recorre nuevamente el texto en su orden original y devuelve el primer carácter cuya frecuencia sea **1**.

Hay dos recorridos, pero no están anidados. El primero cuesta $O(n)$ y el segundo también $O(n)$. Por lo tanto, el costo total es $O(n) + O(n) = O(2n)$, que en notación Big O se expresa como **$O(n)$** . Además, `get()`, `put()` y `getOrDefault()` de `HashMap` tienen costo promedio $O(1)$. Esto es diferente del algoritmo original, donde había un `for` dentro de otro `for`. En ese caso se produce $n \times n = n^2$ y la complejidad es **$O(n^2)$** .

2. Análisis de Colisiones: Si se utiliza un `HashMap` internamente, explique qué ocurre si dos caracteres distintos generan el mismo `hashCode()` (Colisión de Hash) y cómo la estructura resuelve este problema sin perder información.

Una colisión ocurre cuando dos claves distintas terminan asociadas al mismo bucket. Que dos claves tengan el mismo hash no significa que sean iguales. El hash se utiliza para ubicar el bucket candidato y luego **`equals()`** permite distinguir las claves que se encuentran allí.

Por ejemplo, si dos claves distintas producen el mismo hash, conceptualmente el bucket puede contener ambas entradas. Al buscar una clave, **`HashMap`** calcula su hash, llega al bucket correspondiente y compara las claves mediante **`equals()`** hasta identificar la entrada correcta. Por eso una colisión no implica pérdida de información.

Consigna 4: Caso Teórico de Borde y Colisiones

1. El `HashSet` va a contener 2 elementos. Aunque ambas cadenas generan el mismo `hashCode()` (2236), eso no significa que Java las considere iguales. Lo que pasa internamente es lo siguiente:

Cuando hacemos `set.add(s1)` con el String "FB", el `HashSet` calcula `s1.hashCode()`, obtiene 2236, y con ese número determina en qué bucket guardar la referencia. Como el bucket está vacío, la inserta directamente.

Después, cuando hacemos `set.add(s2)` con "Ea", el `HashSet` calcula `s2.hashCode()` y también obtiene 2236. Con ese hash va al mismo bucket donde ya está "FB". Pero acá no termina el proceso: antes de asumir que es un duplicado, el `HashSet` ejecuta `s1.equals(s2)`, es decir, compara `"FB".equals("Ea")`. Como los textos son distintos, el `equals()` devuelve `false`, y entonces Java concluye que son dos objetos diferentes que simplemente colisionaron en el mismo cajón. Por lo tanto, inserta "Ea" en ese mismo bucket junto a "FB".



Esto demuestra lo que dice el contrato formal: que dos objetos tengan el mismo hashCode() no implica que sean iguales. El hashCode() funciona como un filtro rápido de ubicación (te dice en qué cajón buscar), pero la decisión final de igualdad siempre la toma equals(). Si el hash fuera distinto, Java ni se molestaría en llamar a equals() porque buscaría en otro cajón directamente. Pero cuando el hash coincide, obligatoriamente tiene que verificar con equals() para confirmar si realmente es el mismo elemento o si es una colisión.

2. En condiciones normales, las operaciones de un HashMap (put, get, containsKey) tienen un costo promedio de $O(1)$ porque el hashCode() distribuye las claves de forma pareja entre los distintos buckets, y cada bucket tiene pocos elementos (idealmente uno solo).

Pero si se acumulan muchas colisiones, es decir, si muchas claves caen en el mismo bucket, el rendimiento se degrada. En el peor caso teórico imaginable, todas las claves del mapa terminan en un único cajón. Cuando eso pasa, buscar una clave dentro de ese bucket deja de ser instantáneo porque hay que recorrer todos los elementos que están ahí adentro comparando con equals() uno por uno.

En versiones anteriores de Java, los buckets se manejaban internamente como listas enlazadas simples. Si todos los elementos caían en el mismo cajón, la búsqueda dentro de ese cajón era lineal, y el peor caso pasaba de $O(1)$ a $O(n)$, que es básicamente lo mismo que recorrer una lista entera.

A partir de Java 8, cuando un bucket acumula una cantidad considerable de elementos (el umbral es 8 nodos), la lista enlazada interna se transforma automáticamente en un árbol binario balanceado (Rojo-Negro). Gracias a eso, incluso en escenarios de muchas colisiones, la búsqueda dentro de ese bucket baja de $O(n)$ a $O(\log n)$, que es mucho más eficiente. En lugar de revisar las n claves una por una, el árbol permite ir descartando mitades en cada comparación.

el caso ideal es $O(1)$, el peor caso histórico era $O(n)$ con listas enlazadas, y el peor caso en Java moderno es $O(\log n)$ gracias a la conversión automática a árboles balanceados cuando las colisiones se acumulan dentro de un mismo bucket.